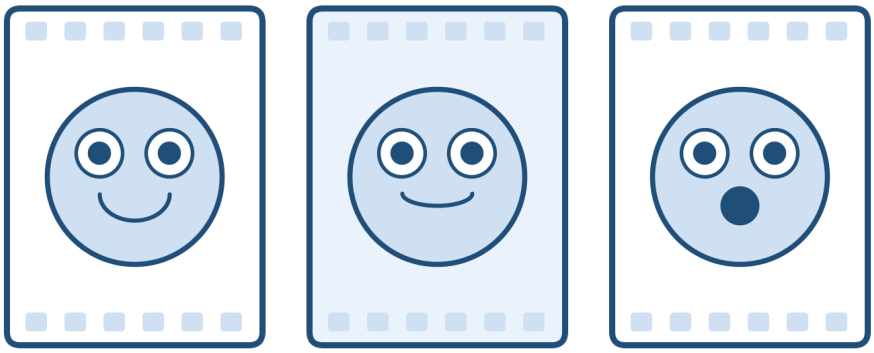
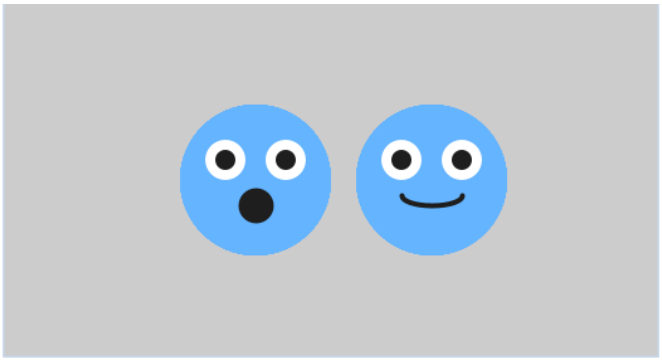
Bonus C - Animate Your Character

Stop sliding a frozen drawing around. Make it move.



pose 0 pose 1 pose 2

Same character. Same moment. Two places.



```
.AddFrameSprite(new MyCharacterFrame(this, width/2 + 70, height/2, 1))
```

Your character moves around the screen, and it's the same drawing the whole time. This chapter makes it come alive: a mouth that talks, legs that walk, wings that flap.

You can take it any time after Chapter 2, once you have a character to animate. It shines most during Chapter 7, when you want your character to feel alive. It pairs well with Bonus A, because picture frames animate the same way. It takes one meeting.

This chapter does it in three moves.

Part	The idea
One	**A flipbook.** An animation is a few still pictures, shown fast, one after another.
Two	**One class, several poses.** Your character, drawn a few slightly different ways.
Three	**Build the flipbook.** An `AnimatedSprite` has the poses, and flips through them for you.

Part One - A flipbook

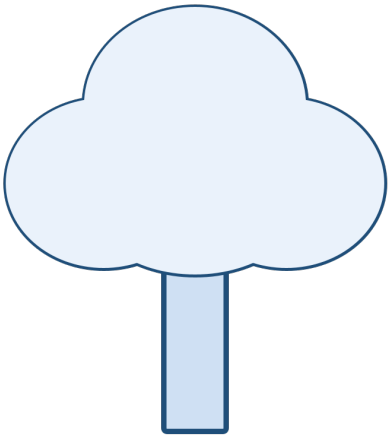
C.1 Pictures, shown fast

Have you ever drawn a flipbook in the corner of a notebook? A cartoon walks the same way: a few still drawings, each a little different, shown fast enough that your eyes see motion.

You met the idea in Chapter 1. `draw()` is already a flipbook: it draws a new page about 60 times a second. Today you add a second flipbook, inside your character, with a few **poses** to flip between.

That means the word *frame* has two jobs today.

- A **game frame** is one run of `draw()`, about sixty a second.
- An **animation frame** is one pose in your character's flipbook.



Part Two - One class, several poses

Step 1 - A frame class

Build it. Make a new tab called `MyCharacterFrame`. Copy your **whole character drawing** from your `MyCharacter` tab into it. Then rename the class, and the constructor, to `MyCharacterFrame`.

Build it. Add a `pose` number, the way the Canvas page's version does:

```
// MyCharacterFrame - ONE POSE of your character.
// It's just like your MyCharacter from Lesson 2, but the "pose" number lets you
// draw a few slightly different versions. AnimatedSprite flips between them.
class MyCharacterFrame extends Blob {

  int pose;  // which version to draw: 0, 1, 2, ...

  MyCharacterFrame(PApplet p, float x, float y, int pose) {
    super(p, x, y, 30, color(100, 180, 255));  // same handshake line as
MyCharacter
    this.pose = pose;
  }

  void drawSprite() {
    // --- Body (same in every pose) ---
    fill(100, 180, 255);
    noStroke();
    ellipse(pos.x, pos.y, 60, 60);

    // --- Eyes (same in every pose) ---
    fill(255);
    ellipse(pos.x - 12, pos.y - 8, 16, 16);
    ellipse(pos.x + 12, pos.y - 8, 16, 16);
    fill(30);
    ellipse(pos.x - 12, pos.y - 8, 8, 8);
    ellipse(pos.x + 12, pos.y - 8, 8, 8);

    // --- Mouth: CHANGES with the pose - this is what makes it animate! ---
    stroke(30);
    strokeWeight(2);
    noFill();
    if (pose == 0) {
      arc(pos.x, pos.y + 6, 24, 18, 0, PI);   // open smile
    } else if (pose == 1) {
      arc(pos.x, pos.y + 6, 24, 8, 0, PI);    // smaller smile
    } else {
      fill(30);
      ellipse(pos.x, pos.y + 10, 12, 12);     // round "talking" mouth
    }
  }
}
```

> **Aside: a property of your own, and `this.pose`**
>
> `int pose;` inside the class gives every frame a number of its own: which pose it draws. It's a property you added, the same way Blob gave your character `pos`. The constructor takes a `pose` too, and `this.pose = pose;` copies it in. Inside a class, `this` means the object itself, so `this.pose` is the frame's own pose, and plain `pose` is the number that was handed in.

Step 2 - Change one thing

Build it. Pick **one** thing on your character to animate. The mouth is easiest. Legs, arms, wings, or ears work well too. Wrap just that part in an `if (pose == 0) { ... } else if (pose == 1) { ... }`, the way the mouth is wrapped above.

> **Aside: `if`, if you're here before Chapter 6**
>
> Chapter 6 is where `if` gets explained properly. For now: `if (pose == 0) { ... }` runs the code in its braces only when `pose` is exactly `0`. `==` asks *exactly equal?* `else if (pose == 1) { ... }` is the next question, asked only when the first answer was no, and a last `else { ... }` catches whatever's left.

Make the difference small. Keep the body and the eyes **exactly** the same in every pose. Only change what should move, or it looks like flickering.

Part Three - Build the flipbook

Step 3 - An `AnimatedSprite`

Build it. In your main tab, your player becomes an **`AnimatedSprite`**, a sprite from the library that has several frames and shows one at a time. Here's the Canvas page's main tab:

```
// Bonus Lesson C - Animate Your Character
import coxprogramming.processing.sprites.*;

AnimatedSprite player;

void setup() {
  size(800, 600);

  // Build the flipbook from several poses - all at the SAME spot so it doesn't
jump.
  // AnimatedSprite shows one pose at a time and flips between them for you.
  player = AnimatedSprite.FromSprite(new MyCharacterFrame(this, width/2,
height/2, 0))
      .AddFrameSprite(new MyCharacterFrame(this, width/2, height/2, 1))
      .AddFrameSprite(new MyCharacterFrame(this, width/2, height/2, 2))
      .setFrameRate(8);   // hold each pose for 8 game frames; smaller = faster

  player.setVelocity(4, 0);  // gives followMouse() a speed to move at
}

void draw() {
  background(240);

  // --- Option A: follow the mouse ---
  player.followMouse();
  player.drawSprite();   // draws the current pose, then turns the page for you

  // --- Option B: arrow-key control ---
  // (comment out Option A above, uncomment these two lines, keep keyPressed
below)
  // player.move();
  // player.drawSprite();
}

// Used with Option B (arrow keys)
void keyPressed() {
  player.keyboardControl();
}
```

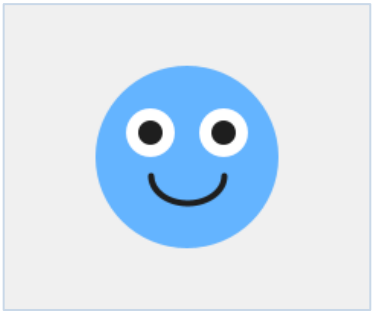
Aside: a chain of calls

`AnimatedSprite.FromSprite(...)` starts the flipbook with its first page. `.AddFrameSprite(...)` adds another page. `.setFrameRate(8)` sets the speed. Each one hands back the flipbook, so the next one can go right on the end. That's why it's one long line, split over four.

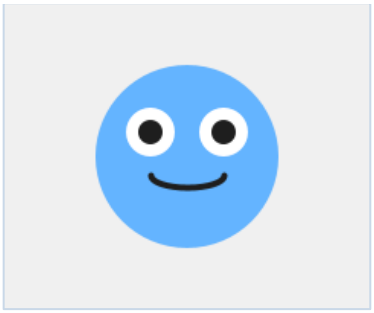
Every frame is made at the same `x` and `y`. That's the crucial part.

Run it.

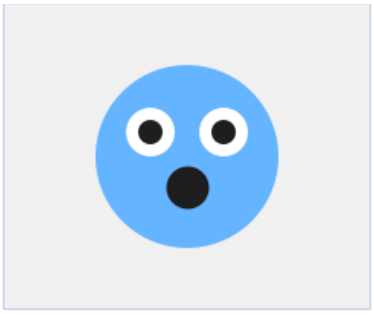
Check. Your character flips through its poses on its own, over and over.



Pose 0, game frames 1 to 8.



Pose 1, game frames 9 to 16.



Pose 2, game frames 17 to 24. Then back to pose 0.

`drawSprite()` draws the pose it's on, and keeps count. Every 8 game frames it turns the page.

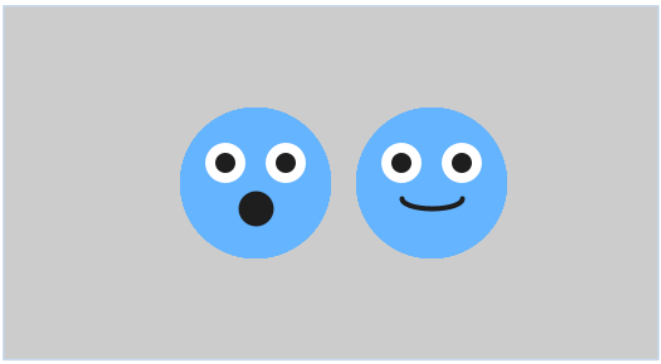
Step 4 - The speed

Build it. Try `setFrameRate(4)`, and then `setFrameRate(20)`. Which speed suits your character?

Check. `4` flips fast and frantic. `20` flips slow and calm.

The number is how many game frames each pose stays on the screen. So a **smaller** number is **faster**. That's backwards from what most people expect.

Watch out. Make one frame at a different `x` or `y`, and your character jumps every time the page turns. Here the second pose was made 70 pixels to the right.



With `background()` switched off: the same character, in two places.

Watch out. Forget `this.pose = pose;`, and every frame's `pose` is `0`. Every page draws pose 0, so nothing seems to animate.

Watch out. Copy the class and forget to rename its constructor, and the sketch stops with a message like this one, on the line that makes the frames:

```
The constructor MyCharacterFrame(...) is undefined
```

The constructor's name has to match the class's name exactly.

Step 5 - Move it

Build it. Your animated character moves exactly like before: `followMouse()` in `draw()`, or `keyboardControl()` in `keyPressed()` with `move()` in `draw()`. The starter has both options, the same way Chapter 4 did.

> **Check.** It moves and animates at the same time, with no extra work.

All the poses move together, because the `AnimatedSprite` carries every one of its frames along with it.

C.5 In your game

Here's the catch. An `AnimatedSprite` isn't a Blob, so it has no `collidesWith()`:

```
The function collidesWith(AnimatedSprite) does not exist.
```

For touching, use **`distanceTo()`**, which every sprite has. It hands back how far away the other sprite is, center to center. So in your Chapter 7 game, each `collidesWith()` becomes a distance check:

```
if (player.distanceTo(coin1) < 40) {   // 40 = how close counts as "touching"
  score = score + 1;
  coin1.moveTo(random(width), random(80, height - 150));
}
```

Pick a number that matches your character's size: about your character's radius plus the coin's.

Two more changes put it in your game. At the top, the player is an `AnimatedSprite`. And in `restartGame()`, the player is built from frames, the way Step 3 builds it, in place of `new MyCharacter(...)`.

> **Check.** Your animated character collects coins and runs into hazards, the same as before.

C.6 Make it yours

Your turn. Add a fourth pose. Try animating **legs or arms** in place of the mouth, and make it walk.

Your turn. Optional: any of these.

- More frames, for smoother motion. More pages make a smoother flipbook.
- A second animated creature, a pet, with its own `setVelocity()`, drifting across the scene.
- Poses by hand. Turn one page on a key, in `keyPressed()`:

```
if (key == 'n') {
  player.nextFrame();          // turn one page
}
```

- Or jump straight to a pose, for a jump or an attack: `player.setFrameNumber(2);`
- Picture frames. A frame can be an `ImageSprite` from Bonus A, so a set of drawings animates the same way: `AnimatedSprite.FromSprite(new ImageSprite(this, x, y, "walk0.png"))`, and `.AddFrameSprite(...)` for the rest.

Chapter summary

You built	It's an example of
`MyCharacterFrame`, with `int pose;`	one class, several poses, and a property of your own
`if (pose == 0) ... else if (pose == 1) ...`	changing one thing between poses
`AnimatedSprite.FromSprite(...).AddFrameSprite(...)`	a flipbook, built in a chain
`setFrameRate(8)`	game frames per pose: smaller is faster
`player.distanceTo(coin1) < 40`	touching, for a sprite that isn't a Blob

New syntax in this chapter

Syntax	Where you met it
a property of your own, `int pose;`	Step 1
`this.pose = pose;`	Step 1
`AnimatedSprite`, `FromSprite()`, `AddFrameSprite()`	Step 3
a chain of calls	Step 3
`setFrameRate()`	Step 4
`distanceTo()`	C.5
`nextFrame()`, `setFrameNumber()`, `key`	C.6

Vocabulary

animation frame -- one pose in a character's flipbook. (C.1)

`AnimatedSprite` -- a sprite from the library that has several frames and shows one at a time. (Step 3)

`distanceTo()` -- how far away another sprite is, center to center. (C.5)

game frame -- one run of `draw()`, about sixty a second. (C.1)

pose -- one version of your character's drawing. (C.1)

Check your understanding

1. What's the difference between a game frame and an animation frame?
2. What does `setFrameRate(4)` do, compared to `setFrameRate(20)`?
3. Your character jumps every time the pose changes. What do you check?
4. Why do the body and eyes have to be exactly the same in every pose?
5. Why can't an `AnimatedSprite` use `collidesWith()`? What does it use?